GitHub Actions · checkout@v7 · Node 26 LTS

CI/CD: desplegar un viernes sin miedo



Del deploy a mano con los dedos cruzados a un pipeline que prueba cada PR, pide aprobación y sabe volver atrás.

Chamba Lab · 2026



CI/CD

Una cinta que prueba y entrega cada cambio.

- **CI:** cada PR se compila y se prueba solo, en una máquina limpia
- **CD:** lo que pasa las pruebas queda listo para producción
- **GitHub Actions:** el motor que lo corre, dentro de tu repo

CI, entrega y despliegue: tres niveles

Integración continua

Cada push o PR se integra con `main` y se prueba en automático.

`npm test` `npm run lint` `build`

LO MÍNIMO EN TODO EQUIPO

Entrega continua

`main` siempre está lista para salir; a producción pasa con una aprobación.

`environment: production` `revisora`

LA MÁS COMÚN

Despliegue continuo

Cada merge a `main` llega a producción sin que nadie toque un botón.

`merge → prod`

EXIGE TESTS DE CONFIANZA

Anatomía de un workflow: el CI de chamba-pedidos

.github/workflows/ci.yml

```yaml
name: CI
on:
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with:
          node-version: 26
      - run: npm ci
      - run: npm test
```

on: cuándo corre. Aquí, en cada PR hacia `main`.

jobs · runs-on: qué trabajos hay y en qué máquina corre cada uno.

steps · uses: una action ya hecha, con su versión fijada (@v7).

steps · run: un comando tuyo, igual que en tu terminal.

Quién participa: cinco piezas alrededor del YAML

Eventos

`push, pull_request, workflow_dispatch`

Runners

`ubuntu-latest, self-hosted`

Workflow

`ci.yml`

Actions del Marketplace

`actions/checkout@v7, actions/setup-node@v7`

Secrets y environments

`secrets.DEPLOY_TOKEN, production`

El evento lo dispara, el runner lo ejecuta, las actions ahorran pasos y los secretos viajan en sobre cerrado.

Runners: la máquina que corre tu workflow



GITHUB-HOSTED	Una máquina virtual **nueva por cada job**: `ubuntu-latest`, `windows-latest` o `macos-latest`. Al terminar, se borra.
SELF-HOSTED	Tu propio servidor con el agente de GitHub. Para hardware especial o red interna; GitHub lo recomienda **solo en repos privados**.
POR QUÉ CURA	Nadie tiene tu `node_modules` ni tu `.env`: si pasa ahí, no fue suerte de tu laptop. Adiós a «en mi máquina funciona».

> **Ojo:** cada job empieza vacío. Por eso todos repiten `checkout` y `setup-node`.

Cómo funciona (1/2): del PR al check



Hasta aquí, **integración continua**: nadie despliega nada, solo se prueba.

Cómo funciona (2/2): del merge a producción



Ahora sí, **entrega continua**: el mismo commit probado es el que llega a producción.

Jobs en paralelo: needs pone el orden



- Sin needs, los jobs corren en paralelo
- Si uno falla, los que lo necesitan se saltan
- Cada job es otra máquina: repite el setup

```yaml
jobs:
  lint:
    runs-on: ubuntu-latest
    steps: # checkout, setup, npm ci
      - run: npm run lint
  test:
    runs-on: ubuntu-latest
    steps: # checkout, setup, npm ci
      - run: npm test
  build:
    needs: [lint, test]
    runs-on: ubuntu-latest
    steps: # checkout, setup, npm ci
      - run: npm run build
```

build arranca solo si lint y test pasan.

Caché de dependencias: no bajes npm cada vez

2.ª corrida

restaura la caché de npm en vez de descargar todo

- La llave sale del hash de `package-lock.json`
- Cambias el lock → caché nueva
- Guarda la caché de npm, no `node_modules`
- Sin uso 7 días, se borra (10 GB por repo)

```yaml
steps:
  - uses: actions/checkout@v7
  - uses: actions/setup-node@v7
    with:
      node-version: 26
      cache: npm
  - run: npm ci
  - run: npm test
```

Una línea: setup-node guarda y restaura la caché.

Secrets y environments: la llave se abre al aprobar

.github/workflows/deploy.yml · job deploy

```yaml
deploy:
  needs: [build]
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment:
    name: production
    url: https://pedidos.example.com
  steps:
    - uses: actions/checkout@v7
    - uses: actions/setup-node@v7
      with: { node-version: 26, cache: npm }
    - run: npm ci && npm run deploy
      env:
        DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
```

Secret: va cifrado y GitHub lo tapa en los logs. Nunca en el YAML.

environment: `production` puede pedir revisoras (hasta 6; basta que apruebe una).

Antes de aprobar, el job no ve los secretos del environment.

Repo privado: revisoras y secretos de environment piden plan Pro o Team.

Checks obligatorios: rojo no entra a main



DÓNDE	Settings → Rules → **Rulesets**, apuntando a la rama `main`.
PULL REQUEST	**Require a pull request before merging**: nadie sube directo a `main`.
STATUS CHECKS	**Require status checks to pass**: eliges el check `test`; si está rojo, el merge se bloquea.
HISTORIA	**Block force pushes**: nadie reescribe lo que ya está en `main`.

El check no es desconfianza: es una QA que nunca se cansa ni se va temprano el viernes.

Entrega o despliegue continuo: el YAML casi no cambia

Entrega continua

```yaml
deploy:
  needs: [build]
  environment: production
  # production tiene revisoras:
  # el job espera un «Approve»
```

Un humano decide cuándo sale

Despliegue continuo

```yaml
deploy:
  needs: [build]
  environment: production
  # production sin revisoras:
  # cada merge a main sale solo
```

Los tests deciden solos

Rollback: volver a la última versión buena



REVERT	`git revert` del PR malo, en un PR nuevo: pasa por CI y deploy como cualquier cambio. **La opción por defecto.**
RE-RUN	En Actions, abre el último deploy bueno y re-ejecuta su job: usa **el mismo commit**. Hasta 30 días después.
LO QUE NO VUELVE	Una migración de base de datos no se deshace con un redeploy: diséñala para que la versión anterior siga funcionando.

Ensaya el rollback un martes. Un sábado a las 2 a. m. no es momento de estrenarlo.

Viernes, 17:48: «es un cambio chiquito»



Deploy a mano desde la laptop: código sin commit, sin tests y sin registro de qué se subió. No hay a dónde volver.

```
vie 17:48  $ git status
           modified: src/precios.js
vie 17:49  # ¿npm test? «Es un cambio chiquito»
vie 17:51  $ scp -r . deploy@prod:/srv/pedidos
vie 17:52  $ ssh deploy@prod "pm2 restart pedidos"
vie 17:53  «En mi máquina funciona.» Rakim se va.
vie 18:05  Prod: todos los pedidos cobran S/ 0.00
sáb 02:10  ¿Qué versión había antes? Nadie sabe.
```

El mismo viernes, con pipeline



VIE 17:48	Rakim sube su rama y abre el **PR #42** «Ajusta precios».
VIE 17:51	Check `test` en rojo: `pedidos.test.js` esperaba S/ 18.90 y recibió S/ 0.00. **El merge se bloquea.**
VIE 17:55	Lo corrige o lo deja para el lunes. Producción ni se enteró.

Sin miedo no es sin riesgo: es que todo lo que sale pasó por la misma máquina limpia, los mismos tests y una aprobación.

Matriz de decisión: cómo llegar a producción

	Deploy a mano	Entrega continua	Despliegue continuo
Si un cambio rompe algo	Lo descubre el cliente	Lo frena el check o la revisora	Lo frenan solo los tests
Velocidad a producción	Rápida, hasta que falla	Minutos, más la aprobación	Minutos después del merge
Qué exige al equipo	Memoria y suerte	CI en cada PR y un environment	Tests en los que confías de verdad

Empieza con entrega continua: check obligatorio en cada PR y producción con aprobación. Despliegue continuo, cuando tus tests atrapen los bugs solos. A mano, nunca más.

Lo que te llevas

- CI en cada PR y check obligatorio: nada entra a main en rojo
- Producción detrás de un environment con aprobación y secretos
- Un rollback ensayado: revert o re-run del último deploy bueno

```
> docs: docs.github.com/actions
> sintaxis: workflow-syntax
> environments: deployments-and-environments
> repo: github.com/Chamba-Lab
```